

95. RegExp aur String ke Methods

`str.match(regex)`

3 modes:

1. **[g] ke bina:** pehla match array me, capturing groups aur properties `index` (position), `input` ke saath.

```
js

let result = "I love JavaScript".match(/Java(Script)/);

result[0];      // JavaScript (poora match)
result[1];      // Script (pehla group)
result.length;  // 2
result.index;   // 7
result.input;   // I love JavaScript
```

2. **[g] ke saath:** saare matches ki array (strings), **groups aur details ke bina.**
3. **Koi match nahi:** `[g]` ho ya na ho, `null` (khaali array nahi). Bhoolne par error: `result.length` par "Cannot read property 'length' of null". Hamesha array chahiye to: `str.match(regex) || []`.

`str.matchAll(regex)`

`match` ka "naya aur behtar" variant (purane browsers ke liye polyfill). Mukhyatah saare matches **groups ke saath** dhundhne ke liye. `match` se 3 farak:

1. Array nahi, **iterable object** (`Array.from` se array banao).
2. Har match **groups ke saath array** (bina `[g]` ke `match` jaisa format).
3. Koi result nahi to `null` nahi, **khaali iterable**.

```
js

let matchAll = '<h1>Hello, world!</h1>'.matchAll(/<(.*?)>/g);
matchAll = Array.from(matchAll);

let firstMatch = matchAll[0];
firstMatch[0];      // <h1>
firstMatch[1];      // h1
firstMatch.index;    // 0
firstMatch.input;    // <h1>Hello, world!</h1>
```

`for..of` me `Array.from` nahi chahiye.

`str.split(regex|substr, limit)`

String ko regex (ya substring) se todta hai:

js

```
'12-34-56'.split('-'); // ['12', '34', '56']  
'12, 34, 56'.split(/,\s*/); // ['12', '34', '56']
```

`str.search(regex)`

Pehle match ki **position** ya `-1`:

js

```
"A drop of ink may make a million think".search(/ink/i); // 10
```

Limitation: sirf **pehla** match. Aage ke matches chahiye to `matchAll`.

`str.replace(str|regex, str|func)`

Search-replace ka "Swiss army knife". Regex ke bina bhi chalta hai:

js

```
'12-34-56'.replace("-", ":"); // 12:34-56
```

Pitfall: pehla argument **string** ho to sirf **pehla** match replace hota hai. Sab ke liye regex `/-/g` (`g` zaruri):

js

```
'12-34-56'.replace(/-/g, ":"); // 12:34:56
```

Replacement string me special combinations:

Symbols	Kaam
<code>\$&</code>	poora match
<code>\$`</code>	match se pehle ka hissa
<code>\$'</code>	match ke baad ka hissa
<code>\$n</code>	<code>(n)</code> (1-2 digit) wale group ka content
<code>\$<name></code>	naam wale group ka content
<code>\$</code>	<code>(.)</code> character

```
js
```

```
"John Smith".replace(/(john) (smith)/i, '$2, $1'); // Smith, John
```

"Smart" replacements ke liye doosra argument function ho sakta hai. Har match par call hota hai aur uski return value replacement banti hai. Arguments: `func(match, p1, p2, ..., pn, offset, input, groups)`:

1. `match`: match
2. `p1..pn`: capturing groups ka content (agar hain)
3. `offset`: match ki position
4. `input`: source string
5. `groups`: named groups ka object

Parentheses na hon to sirf 3: `func(str, offset, input)`.

```
js
```

```
"html and css".replace(/html|css/gi, str => str.toUpperCase()); // HTML and CSS
"Ho-Ho-ho".replace(/ho/gi, (match, offset) => offset); // 0-3-6
"John Smith".replace(/(\w+) (\w+)/, (match, name, surname) => `${surname}, ${name}`);
```

Kai groups ho to rest parameters: `(...match) => \`${match[2]}, ${match[1]}`` `. Named groups ho to` `groups`` object **hamesha aakhri** argument hota hai:

```
js
```

```
"John Smith".replace(/(?<name>\w+) (?<surname>\w+)/, (...match) => {  
  let groups = match.pop();  
  return `${groups.surname}, ${groups.name}`;  
});
```

Function se poori taqat milti hai: match ki saari info, bahar ke variables ka access, kuch bhi.

`str.replaceAll(str|regexp, str|func)`

`replace` jaisa, 2 bade farak:

1. Pehla argument **string** ho to **saare** occurrences replace hote hain (`replace` sirf pehla).
2. Pehla argument `g` ke bina **regexp** ho to **error**. `g` ke saath `replace` jaisa.

js

```
'12-34-56'.replaceAll("-", ":"); // 12:34:56
```

`regexp.exec(str)`

Regex par call hota hai (string par nahi). `g` ke bina `str.match(regexp)` jaisa. `g` ke saath:

- Pehla match deta hai aur uske turant baad ki position `regexp.lastIndex` me rakhta hai.
- Agli call `lastIndex` se search shuru karti hai, agla match deta hai aur `lastIndex` update.
- Match nahi mila to `null` aur `lastIndex = 0`.

Purane zamane me `matchAll` se pehle loop me sab matches groups ke saath isi se milte the:

js

```
let str = 'More about JavaScript at https://javascript.info';  
let regexp = /javascript/ig;  
let result;  
  
while (result = regexp.exec(str)) {  
  alert(`Found ${result[0]} at position ${result.index}`);  
}
```

`lastIndex` khud set karke kisi position se search:

js

```
let regexp = /\w+/g;
regexp.lastIndex = 5;
regexp.exec('Hello, world!'); // world
```

Agar `y` flag ho to search **exactly** `lastIndex` par hoti hai (na ki wahan se aage):

```
js

let regexp = /\w+/y;
regexp.lastIndex = 5;
regexp.exec('Hello, world!'); // null (position 5 par comma hai)
```

`regexp.test(str)`

Match hai ya nahi, `true/false`:

```
js

/love/i.test("I love JavaScript"); // true
"I love JavaScript".search(/love/i) != -1; // true (same)
```

`g` flag ho to `regexp.test` bhi `lastIndex` se dhundhta hai aur use update karta hai (bilkul `exec` jaisa), to kisi position se search ke liye use ho sakta hai:

```
js

let regexp = /love/gi;
regexp.lastIndex = 10;
regexp.test("I love JavaScript"); // false
```

Pitfall: same global regexp ko alag strings par baar baar test karna

`test` `lastIndex` badhata hai, to doosri string par search non-zero position se shuru ho sakti hai:

```
js

let regexp = /javascript/g;
regexp.test("javascript"); // true (lastIndex = 10 ab)
regexp.test("javascript"); // false
```

Fix: har search se pehle `regexp.lastIndex = 0` karo, ya regexp ke methods ke bajay string methods (`str.match/search/...`) use karo, jo `lastIndex` use nahi karte.